

轨迹预测程序PathPrediction部署

1、总体架构：

1.1、接收自定义ros消息：

TerrainMapComplex.msg

```
float64 local_time          # in ms
float64 UTC_time            # in ms
int32  message_num

Pose6D globalPoseStamped
geometry_msgs/Pose2D localPoseStamped  # freezed coordination: localpose

float32[250000] elevation_mu
float32[250000] elevation_sigma
float32[250000] elevation_maxmin
float32[250000] elevation_predict
float32[250000] elevation_slope

geometry_msgs/Point32[20] path_past
float32 lidar_h_offset      # lidar_h_offset = lidar_h_cur - lidar_h_first
float64 residual_x
float64 residual_y
```

Pose6D.msg为

```
float64 local_time
float64 x
float64 y
float64 z
float64 azimuth
float64 pitch
float64 roll
```

文件位置：UGV_2023/src/message/world_state/msg

1.2、模型推理

python程序模型保存：

```
model = torch.jit.trace(model,(feature_map_batch, past_path_batch,
mask_road_batch, class_batch), strict=False)
model = torch.jit.optimize_for_inference(model)
model.save("pathmodel_hexi.pt")
```

1.3、ros发出预测的20个轨迹点

```
torch::Tensor predict_waypoint = output.toTuple()->elements()[0].toTensor();
// 遍历 predict_waypoint 并填充 waypoints 消息
for (int i = 0; i < predict_waypoint.size(1); ++i)
{
    geometry_msgs::Point32 point;
    point.x = predict_waypoint[0][i][0].item().toFloat();
    point.y = predict_waypoint[0][i][1].item().toFloat();
    point.z = 0.0;
    // std::cout << "x:" << point.x << "y:" << point.y << std::endl;
    path_predict.points.push_back(point);
}
// 一次性发布所有点
ros_pub_path.publish(path_predict);
```

2、超参数设置：

2.1、图片大小：

```
const int width = 300;
const int height = 300;
const int Mask_SIZE = 95;
const float RESOLUTION = 0.2;
const float INVALID_Z = -123456.0; // 无效的Z值
```

2.2、卫星图位置，参数：

```
std::string satellite_path = "./Satellite_Map";

std::string img_path = satellite_img_path + "/Hexi_gl/Hexi_gl.png";
std::string mask_path = satellite_img_path + "/Hexi_gl/Hexi_gl_mask.png";
top_left_blh.x = 112.86814928054808;
top_left_blh.y = 28.107692088513648;
low_right_blh.x = 112.87246763706206;
low_right_blh.y = 28.10562665626037;
```

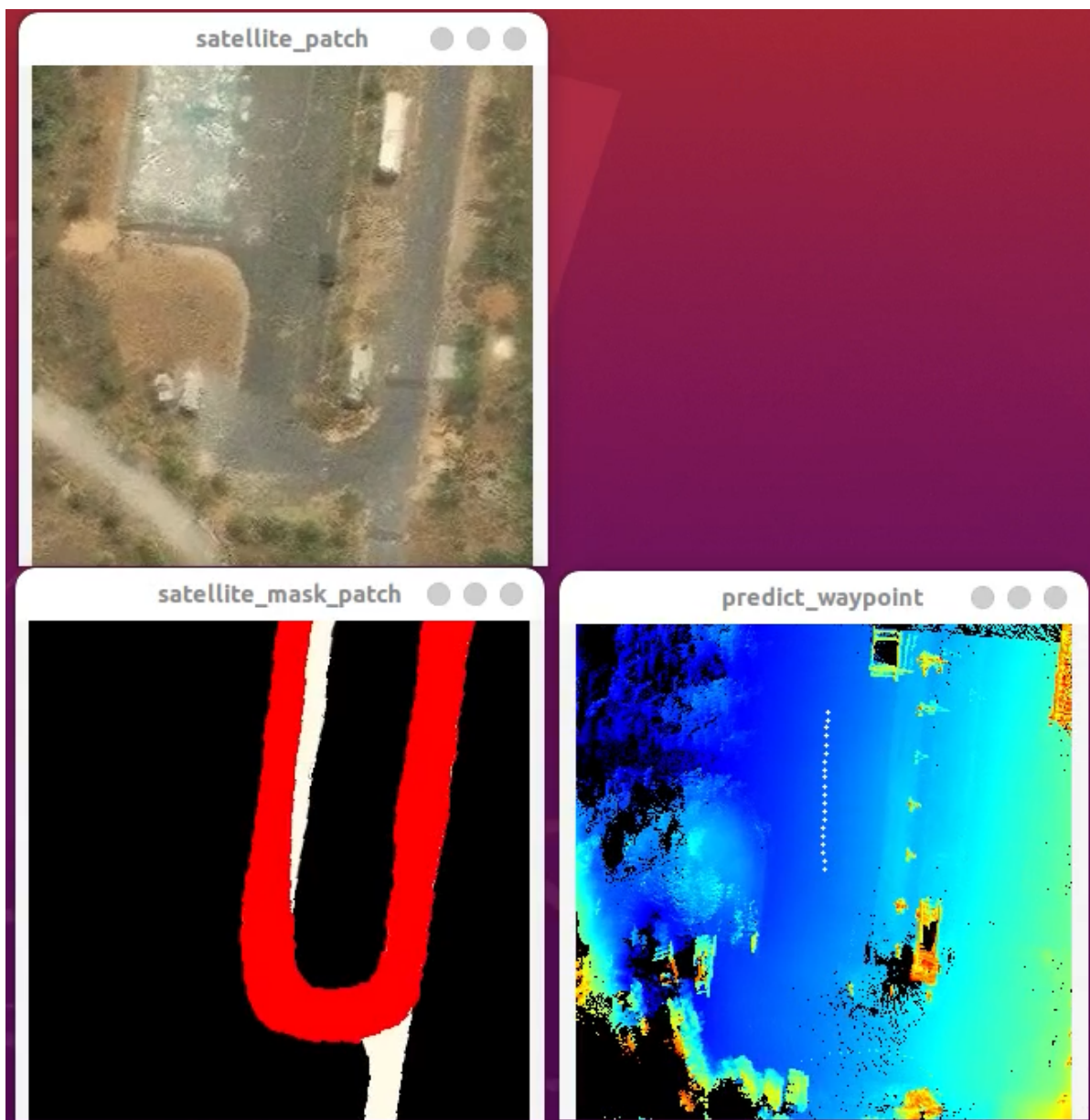
2.3、图片截取（输入为500X500），暂时使用300X300，根据需要修改

```
torch::Tensor tensor_elevation_mu = ExtractSubtensor(msg->elevation_mu, 300,
300);
```

2.4、模型放置

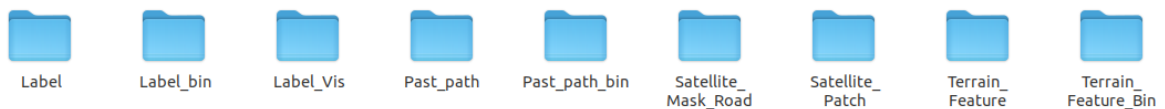
Satellite_Map文件夹和模型放在/home/guanglei/UGV_2023/program/module_modeling/bin

可视化效果



Python程序训练

训练数据集：



数据示例: <https://pan.quark.cn/s/15434d95fb65>

train.7z

dataloader用到的数据

Label_bin	未来路径轨迹点30*2
Past_path_bin	历史路径轨迹点20*2
Satellite_Mask_Road	全局路径掩膜图
Terrain_feature_bin	地形特征

模型推理输入：

```
◦ feature_map_batch[1,5,300,300]
◦ past_path_batch[1,20,2]
◦ mask_road_batch[1,3,95,95]
◦ class_batch[1]
```

输出：

```
predict_waypoint[1,20,2]
```

训练数据生成：

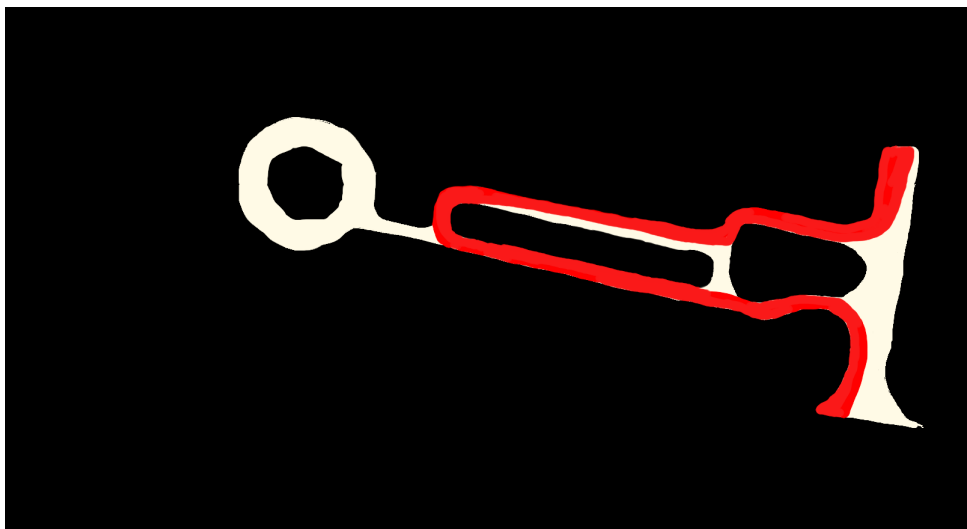
c++程序Terrain_Analysis

生成用于轨迹预测的地形模型的步骤：

1. 在 `./bin/parameters/Dataloader.ini` 参数文件中更改数据路径，位姿读取方式（惯导 or SLAM结果）；
2. 将作业场景的卫星地图以及从卫星图上标注得到的路网和全局路掩膜图放在 `./bin/Satellite_Map` 文件夹下，图像场景格式即可，同时需要在 `Satellite_Patch` 类的构造函数中指定卫星图的路径以及卫星图的左上角和右下角经纬度坐标：

```
std::string img_path = satellite_img_path + "/Hexi_g1/Hexi_g1.png";
top_left_blh.x = 112.86814928054808;
top_left_blh.y = 28.107692088513648;
low_right_blh.x = 112.87246763706206;
low_right_blh.y = 28.10562665626037;
```

路网图示例如下：



3. 在 `bool distance_flag = preprocess->distance_filter_SLAM(preprocess->scan_terrain.Tr_w_l)` 函数中指定关键帧采样距离，一般直线采样距离大一些（5m），弯道采样距离小一些（0.5m）；需要在 `if(distance_flag && frame_idx > 8130 && frame_idx <= 8210)` 中修改起始帧号，一般先看一下可视化结果，剖除开头和结尾的若干帧，等车辆开始运动形成稳定的地形图后再开始保存地形模型，同样结尾也需要在车辆停止前结束保存地形模型。另外，对于整个数据集，可以以5m的采样距离全部跑一遍，之后再以0.5m的采样距离对弯道数据重新生成一遍。

4. 最后输出包括六个文件，分别在 `NDT_Mapper::BGK_Terrain_Feature()` 和 `Satellite_Patch::Run()` 函数中：

```
// NDT_Mapper::BGK_Terrain_Feature()
char output_name[300], label_name[300], label_vis_name[300],
past_path_name[300];
sprintf(output_name, "./Satellite_Terrain_data/KITTI/07/Terrain_Feature/Terrain_map_%06d.txt", ID); //地形图
sprintf(label_name,
"./Satellite_Terrain_data/KITTI/07/Label/path_label_%06d.txt", ID); //轨迹真值
sprintf(label_vis_name, "./Satellite_Terrain_data/KITTI/07/Label_Vis/path_label_vis_%06d.png", ID); //地形、轨迹可视化
sprintf(past_path_name, "./Satellite_Terrain_data/KITTI/07/Past_path/past_path_%06d.txt", ID); //过去轨迹

// Satellite_Patch::Run()
char patch_output_name[200], mask_output_name[200];
sprintf(patch_output_name, "./Satellite_Terrain_data/KITTI/08/Satellite_Patch/satellite_patch_%06d.png", ID); // 卫星图块
sprintf(mask_output_name,
"./Satellite_Terrain_data/KITTI/08/Satellite_Mask_Road/satellite_road_%06d.png", ID); // 全局路图块
```

Libtorch安装：

参考<https://zhuanlan.zhihu.com/p/92374964>

下载网址，根据cuda版本更改 cu111

<https://download.pytorch.org/libtorch/cu111>

根据pytorch版本，选择页面链接进行下载[libtorch-cxx11-abi-shared-with-deps-1.8.1%2Bcu111.zip](#)